LegalEase

AI-Powered Legal Document Drafting Assistant

Project Documentation

A complete technical and functional documentation based on the supplied project-documentation reference, adapted to the LegalEase application.

Project	LegalEase
Application Type	Generative AI Web Application
Frontend	Streamlit
Backend	FastAPI + Uvicorn
AI/LLM	Google Gemini API
Document Outputs	TXT, DOCX, PDF
Operating Mode	Demo Mode / Gemini API Mode
Documentation Date	26 September 2026

Important: LegalEase produces AI-generated document drafts. It is not a substitute for legal advice or professional legal review.

Contents

1. Introduction

2. Project Overview

3. Problem Statement

4. Objectives

5. Key Features

6. System Architecture

7. End-to-End Workflow

8. Technology Stack

9. Project Folder Structure

10. AI / Gemini Integration

11. Demo Mode and Gemini API Mode

12. Document Generation and Export

13. User Interface

14. Backend and API Layer

15. Configuration and Environment Variables

16. Installation and Setup

17. Running the Application

18. Testing Procedure

19. Error Handling and Troubleshooting

20. Security and Privacy Considerations

21. Limitations

22. Future Enhancements

23. Conclusion

1. Introduction

LegalEase is a Generative AI-powered web application designed to help users create structured legal-document drafts from information supplied through a web interface. The system combines a Streamlit frontend, a FastAPI backend, a dedicated AI generation layer, and document-export components.

The supplied reference document uses a modular project-report structure covering introduction, overview, architecture, setup, folder structure, APIs, interface, conclusion, and future enhancements. This document follows that same style while replacing the reference project's education-focused content with the actual LegalEase architecture and workflow.

The application supports two AI-generation configurations. Demo Mode allows the software pipeline to be demonstrated without contacting Gemini. Gemini API Mode connects the application to Google's Gemini service so that the document text is generated dynamically by a cloud LLM.

2. Project Overview

LegalEase is intended as a practical demonstration of how Generative AI can be integrated into a full-stack Python application. A user supplies document type, parties, terms, dates, jurisdiction, and optional instructions. The backend validates the request, constructs a controlled prompt, invokes the configured generation mode, and returns the generated draft to the frontend.

Component	Role
Streamlit frontend	Collects user inputs, displays status/results, and provides download controls.
FastAPI backend	Provides the application API and coordinates validation, generation, and export.
AI generation layer	Abstracts Gemini and demo generation so the rest of the application remains modular.
Google Gemini API	Provides cloud-based generative AI when Gemini API Mode is enabled.
Export layer	Converts generated content into TXT, DOCX, and PDF outputs.
.env configuration	Stores API key and runtime configuration without hard-coding secrets.

3. Problem Statement

Preparing structured legal-document drafts manually can require repetitive formatting and clause-writing work. Users may also find it difficult to turn scattered information into a consistent document structure.

LegalEase addresses this software-engineering problem by providing a single interface where structured information can be collected and passed through an AI-assisted drafting pipeline. The generated output is intended as a draft for review, not as a guaranteed legally sufficient document.

4. Objectives

- Provide a simple web interface for entering legal-document requirements.
- Use Generative AI to produce dynamically worded document drafts.
- Keep the AI provider isolated behind a dedicated generation module.
- Support a Demo Mode for development and demonstrations without API calls.
- Support Gemini API Mode for real LLM generation.

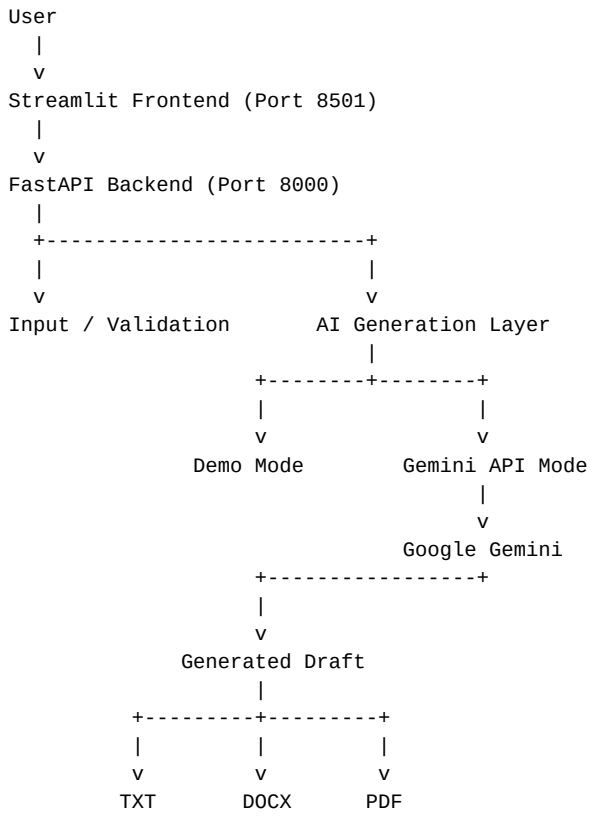
- Provide downloadable TXT, DOCX, and PDF outputs.
- Keep API credentials outside source code through environment configuration.
- Provide a modular architecture that can be extended with additional document types and AI providers.
- Handle temporary AI-service failures without crashing the entire application.

5. Key Features

- Document drafting: Generates a structured draft from user-provided information.
- Configurable AI: Switches between Demo Mode and Gemini API Mode.
- Structured input: Supports document type, parties, terms, dates, jurisdiction, and special instructions.
- Multiple exports: Download generated content as TXT, DOCX, or PDF.
- Local development: Runs on a Windows development machine using a Python virtual environment.
- Separation of concerns: Frontend, backend, AI logic, configuration, and export functionality are kept separate.

6. System Architecture

The application follows a layered architecture similar in principle to the supplied reference project, but its responsibilities are adapted to legal-document drafting.



This design allows the user interface and export pipeline to remain largely unchanged when switching between demo generation and real Gemini generation.

7. End-to-End Workflow

Step	Operation
1	User opens the Streamlit application.
2	User selects or enters the required document information.
3	Streamlit sends the request to the FastAPI backend.
4	FastAPI validates and normalizes the input.
5	The AI generation layer checks DEMO_MODE and the configured Gemini key/model.
6	In Demo Mode, a local demonstration response is returned.
7	In Gemini API Mode, a prompt is sent to the configured Gemini model.
8	The generated draft is returned to the backend.
9	The frontend displays the generated content and exposes download options.
10	The user can download TXT, DOCX, or PDF output.

8. Technology Stack

Technology	Purpose
Python 3.10+	Application runtime.
Streamlit	Interactive frontend and user workflow.
FastAPI	Backend REST API and application orchestration.
Uvicorn	ASGI server used to run FastAPI.
Google GenAI SDK	Python integration with Google's Gemini API.
python-dotenv / environment configuration	Loads runtime values from .env.
DOCX generation library	Creates Microsoft Word-compatible output.
ReportLab	Creates PDF output.
Virtual environment (.venv)	Isolates project dependencies.

9. Project Folder Structure

The working LegalEase structure is organized so that frontend, backend, AI logic, exports, tests, and configuration can be maintained independently.

```
LegalEase/
■■■ backend/
■   ■■■ app/
■       ■■■ main.py
■       ■■■ config.py
■       ■■■ api/
■       ■■■ ai_core/
■       ■   ■■■ gemini_generator.py
■       ■■■ services/
■■■ frontend/
■   ■■■ app.py
■■■ tests/
■■■ assets/
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
```

```
■■■ README.md
```

The exact implementation can contain additional package files and submodules. The important design principle is that the Gemini integration should remain inside the AI layer instead of being mixed directly into the Streamlit interface.

10. AI / Gemini Integration

Gemini API Mode is the real Generative AI path in LegalEase. The application loads the API key from the environment and creates a Gemini client through the Google GenAI Python SDK. The generator builds a prompt containing the document type, supplied facts, terms, jurisdiction, and any special instructions.

```
GEMINI_API_KEY=your_actual_key
GEMINI_MODEL=your_configured_model
DEMO_MODE=false
```

The API key must never be committed to source control or placed directly inside Python code. The .env file should be excluded through .gitignore.

The Gemini model can return natural-language document text. The application then treats that text as the draft content and passes it to the export layer.

11. Demo Mode and Gemini API Mode

Area	Demo Mode	Gemini API Mode
Google API call	No	Yes
API key required	No	Yes
Internet required for generation	No	Yes
Dynamic LLM generation	No / simulated	Yes
Useful for UI/export testing	Yes	Yes
Consumes Gemini quota	No	Yes
Recommended use	Development, demos, offline testing	AI feature testing and real demonstrations

Keeping both modes is useful because UI and export development can continue without consuming Gemini quota, while Gemini Mode can be enabled when testing the actual AI integration.

12. Document Generation and Export

The generation and export stages are deliberately separated. Gemini does not need to know how the final file is formatted. It only produces the draft text. Dedicated exporters then transform the same draft into different file formats.

```
User input
    |
    v
AI-generated draft text
    |
    +--> TXT
    +--> DOCX
    +--> PDF
```

This separation makes it possible to improve PDF or Word formatting without changing the AI prompt or model integration.

13. User Interface

The Streamlit interface provides the main interaction surface. A typical workflow contains fields for document type, parties, relevant terms, dates or jurisdiction, and optional instructions, followed by a Generate Document action.

After generation, the interface displays the draft and provides download controls. Status messages should clearly distinguish successful generation from API errors such as temporary service unavailability or quota exhaustion.

The interface should also retain a legal-use disclaimer stating that generated documents are drafts and should be reviewed by a qualified legal professional before use.

14. Backend and API Layer

FastAPI is responsible for the server-side application layer. The frontend should communicate with the backend rather than directly embedding Gemini credentials in browser-side code.

Layer / Operation	Responsibility
Health check	Confirms that the backend is running and reports the current operating mode.
Generate document	Accepts document details and invokes the configured generation service.
Export handling	Prepares generated content for TXT, DOCX, and PDF download.
Error handling	Converts provider or validation errors into readable application responses.

The health endpoint is also useful during development because it verifies that the backend has loaded the current environment configuration, including whether Demo Mode is enabled.

15. Configuration and Environment Variables

The following configuration pattern is used for local development:

```
GEMINI_API_KEY=YOUR_REAL_GEMINI_KEY
GEMINI_MODEL=YOUR_CONFIGURED_MODEL
DEMO_MODE=false

BACKEND_URL=http://127.0.0.1:8000
ALLOWED_ORIGINS=http://127.0.0.1:8501,http://localhost:8501
```

For a demonstration without Gemini, set DEMO_MODE=true. For actual Gemini generation, set DEMO_MODE=false and provide a valid API key and supported model.

16. Installation and Setup

Prerequisites: Windows with Python installed, VS Code, internet access for installing packages and using Gemini, and a Google Gemini API key when Gemini Mode is enabled.

Step 1 — Open the project: Open the LegalEase folder in VS Code.

Step 2 — Create the environment (first setup only):

```
python -m venv .venv
```

Step 3 — Activate it:

```
.\.venv\Scripts\Activate.ps1
```

Step 4 — Install dependencies:

```
pip install -r requirements.txt
```

Step 5 — Configure .env: Add the Gemini API key and set DEMO_MODE appropriately.

Step 6 — Optional SDK update:

```
python -m pip install -U google-genai
```

17. Running the Application

Terminal 1 — Backend:

```
cd C:\Projects\LegalEase
.\.venv\Scripts\Activate.ps1
python -m uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000
```

Terminal 2 — Frontend:

```
cd C:\Projects\LegalEase
.\.venv\Scripts\Activate.ps1
python -m streamlit run frontend/app.py --server.port 8501
```

Open the frontend at http://localhost:8501. The backend is normally available at http://127.0.0.1:8000.

To stop: Press Ctrl+C in both terminals. The project files, .venv, and .env remain intact, so installation is not repeated on the next run.

18. Testing Procedure

- Confirm the virtual environment is active.
- Start FastAPI and verify the health endpoint.
- Start Streamlit and open the frontend.
- First test Demo Mode with fictional information to verify the complete UI and export pipeline.
- Switch to Gemini API Mode and generate a short test document.
- Verify that the generated content changes according to the supplied inputs.
- Download and open TXT, DOCX, and PDF outputs.
- Test invalid or incomplete inputs and confirm a readable validation error is displayed.
- Test a temporary Gemini failure and confirm the interface reports the problem without crashing the frontend.
- Never use confidential client data during initial API testing.

19. Error Handling and Troubleshooting

Symptom	Likely cause	Action
Health shows demo_mode=true	Environment setting is still enabled.	Set DEMO_MODE=false and restart FastAPI.
401 authentication error	Invalid/missing API key.	Check GEMINI_API_KEY in .env; do not expose the key.
403 permission error	API/project/key restriction issue.	Check Google AI Studio project/key configuration.

Symptom	Likely cause	Action
404 model error	Configured model is unavailable or incorrect.	Use a model currently available to the API key and update GEMINI_MODEL.
429 error	Rate/quota limit reached.	Wait, reduce request frequency, or check current API quota.
503 UNAVAILABLE	Temporary provider capacity/service issue.	Wait and retry; implement bounded retries for transient failures.
Frontend cannot reach backend	FastAPI is not running or URL mismatch.	Start port 8000 backend and verify BACKEND_URL.

The application should not expose raw API keys in UI error messages or logs.

20. Security and Privacy Considerations

- Store the Gemini API key only in .env or another secure secret-management system.
- Add .env to .gitignore before using Git or GitHub.
- Do not hard-code the key into Streamlit or FastAPI source code.
- Do not paste API keys into screenshots, project reports, public repositories, or chat messages.
- Use fictional data while testing the free-tier API.
- Treat AI-generated legal text as a draft that requires human review.
- If the project is later deployed publicly, add authentication, rate limiting, HTTPS, server-side secret storage, logging controls, and stronger input/output validation.

21. Limitations

- Generated text can contain factual, contextual, or legal inaccuracies.
- The output is not a substitute for advice from a qualified legal professional.
- Gemini API availability and quotas can vary by model and account.
- Free-tier API usage is subject to applicable limits.
- The local Demo Mode is not evidence that the Gemini service is available.
- Public deployment requires additional security and operational controls beyond local development.

22. Future Enhancements

- Add user accounts and document history.
- Add document templates for employment agreements, NDAs, service agreements, leases, and other categories.
- Add multilingual drafting, including English and regional Indian languages.
- Add clause-level editing and regeneration instead of regenerating the entire document.
- Add document comparison and version history.
- Add retrieval-augmented generation using a curated legal knowledge base where legally appropriate.

- Add structured JSON output from the model before rendering the final document.
- Add stronger validation and prompt-injection defenses for uploaded/reference material.
- Add cloud deployment with HTTPS, authentication, secret management, monitoring, and rate limiting.
- Add automated unit, integration, and end-to-end tests in CI.

23. Conclusion

LegalEase demonstrates a complete Generative AI application architecture in which a user-facing Streamlit interface communicates with a FastAPI backend and a dedicated AI generation layer. The application can operate in Demo Mode for development or in Gemini API Mode for real cloud-based LLM generation.

The modular structure makes the project suitable for continued development: the AI provider can be changed independently of the frontend, and the same generated draft can be exported to multiple document formats.

For academic or demonstration use, the project should be presented as an AI-assisted document-drafting system. Any generated legal document should be treated as a draft and reviewed by an appropriately qualified professional before real-world use.

Reference basis: This report follows the organization and documentation style of the supplied “EduGenie Project Documentation” reference, adapting its sections for LegalEase and its Streamlit + FastAPI + Gemini + document-export architecture.